

Classes & Objects

The Building Blocks of C++

Essential Object-Oriented Logic



ATTRIBUTES • METHODS • INSTANTIATION

1. What is a Class?

In C++, a **Class** is a user-defined data type. It acts as a **blueprint** or a psychological template for creating objects.

A class bundles two things together:

- **Data Members (Attributes):** Variables that store information.
- **Member Functions (Methods):** Functions that define behavior.

The Blueprint Analogy

Blueprint (Class)

[Plan for a Smartphone]

↓ Attributes: Color, Brand, Storage

↓ Behaviors: Call(), Text(), Charge()

2. Class Syntax

When defining a class, we use the keyword `class` followed by a name. We also use `public:` to ensure the data is accessible from outside the class.

```
class CodeHive { public: string courseName; // Attribute int duration; //  
Attribute // Method (Function inside a class) void showDetails() { cout <<  
"Course: " << courseName; cout << ", Duration: " << duration << "  
months"; } };
```

Note: Don't forget the semicolon `;` at the end of the class definition. It is a common mistake for beginners!

3. What is an Object?

An **Object** is a real entity created from a class blueprint. While the class is just a plan on paper, the object takes up actual space in the computer's memory.

Actual Phone (Object)

Color: "Midnight Black"

Brand: "Apple"

Storage: 256GB

Think of the Class as the definition of a "Car," and the Object as the specific "Red Ford" currently parked in your driveway.

4. Realization (Instantiation)

To create an object, you use the class name just like a data type (like `int` or `string`) inside your `main()` function.

```
int main() { // 1. Create the object CodeHive cppCourse; // 2. Access
attributes cppCourse.courseName = "C++ OOP"; cppCourse.duration = 6; // 3.
Call methods cppCourse.showDetails(); return 0; }
```

The Dot Operator

We use the  operator to "dot into" the object and access its internal contents.

5. Multiple Instances

One of the greatest strengths of classes is that you can use the same blueprint to create many different objects, each holding its own unique data.

```
CodeHive pythonCourse; pythonCourse.courseName = "Python Pro";  
pythonCourse.duration = 4; CodeHive webCourse; webCourse.courseName = "Web  
Dev"; webCourse.duration = 8;
```

Even though they use the same CodeHive code, `pythonCourse` and `webCourse` are completely independent in memory.

6. Why Use Classes?

1. Organization

Instead of having variables scattered everywhere, you group them logically. All "Course" data stays within the "Course" class.

2. Code Reusability

Once you write a Student class, you can use it to manage 10 students or 10,000 students without writing new code for each one.

3. Real-World Modeling

Programming becomes easier when you think in nouns (Car, User, Account) instead of just complex logical steps.

7. Quick Summary

- **Class:** The Blueprint / Plan.
- **Object:** The real-world thing (Instance).
- **Attributes:** The variables inside a class.
- **Methods:** The functions inside a class.
- **Dot (.):** The tool used to access members.

MODULE COMPLETE

Next Stage: Access Specifiers & Constructors